Blockchain and Decentralization

Lecture 3 — Consensus, FLP, and Paxos

Pierre Sutra

Télécom SudParis

CSC5006

version 1.0

Previously, in lecture 2

- the **mempool**: buffer and disseminate the pending transactions, fee ordering (or arrival order) to select

Previously, in lecture 2

- the **mempool**: buffer and disseminate the pending transactions, fee ordering (or arrival order) to select
- a node keeps and relays only **valid** transactions

Previously, in lecture 2

- the **mempool**: buffer and disseminate the pending transactions, fee ordering (or arrival order) to select
- a node keeps and relays only **valid** transactions
- specifying dissemination means choosing a **model** (links, failures, synchrony, graph) and **properties**: **safety** and **liveness**

Previously, in lecture 2

- the **mempool**: buffer and disseminate the pending transactions, fee ordering (or arrival order) to select
- a node keeps and relays only **valid** transactions
- specifying dissemination means choosing a **model** (links, failures, synchrony, graph) and **properties**: **safety** and **liveness**
  - **reliable broadcast**: relay before delivering

Previously, in lecture 2

- the **mempool**: buffer and disseminate the pending transactions, fee ordering (or arrival order) to select
- a node keeps and relays only **valid** transactions
- specifying dissemination means choosing a **model** (links, failures, synchrony, graph) and **properties**: **safety** and **liveness**
  - **reliable broadcast**: relay before delivering
  - at scale, **gossip**: **anti-entropy** is slow but guaranteed; **rumor mongering** is fast but probabilistic

Where we are

Lectures 1–2	building blocks: cryptography, data dissemination
Lecture 3	**consensus (FLP, Paxos)**
Lecture 4	Byzantine failures
Lecture 5	replicated state machines (PBFT)
Lectures 6–8	blockchains proper: distributed ledgers (Proof-of-X, layer 2), wallets, smart contracts and privacy, DeFi
Lectures 9–11	advanced topics: decentralized storage and compute, CRDTs, cross-chain transactions

Today

How can processes **agree** on a single value, even if some of them crash?

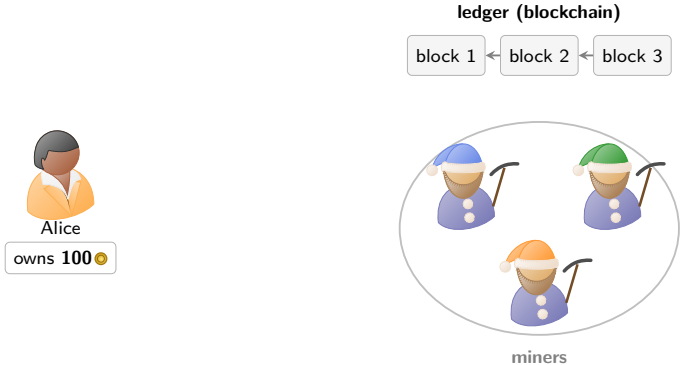
- why a blockchain needs consensus
- the FLP impossibility result
- Paxos: consensus with message passing

Part	Time
Context	8’
FLP	18’
Paxos	28’
Wrap-up	6’



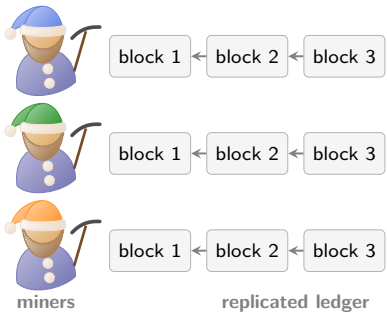
Outline

1. Context: consensus for blockchains
2. The FLP impossibility result
3. Paxos
4. Wrap-up

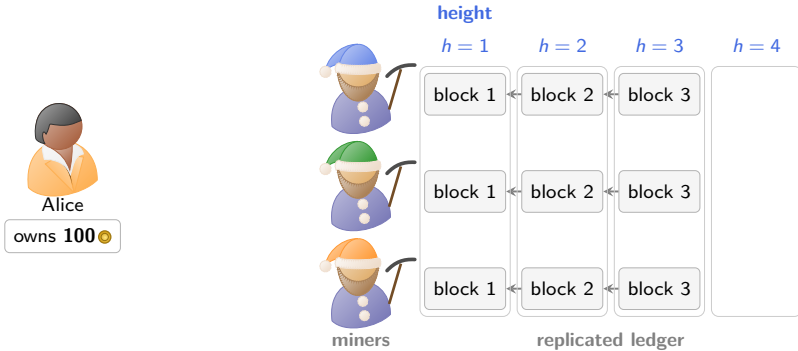


Miners build blocks of transactions, appended to **the** ledger (lectures 1 and 2).

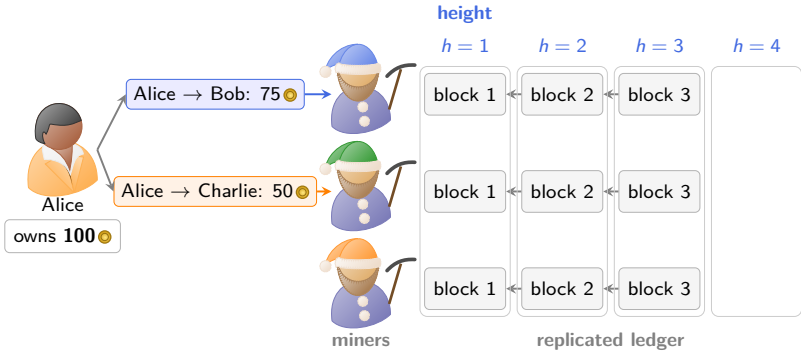




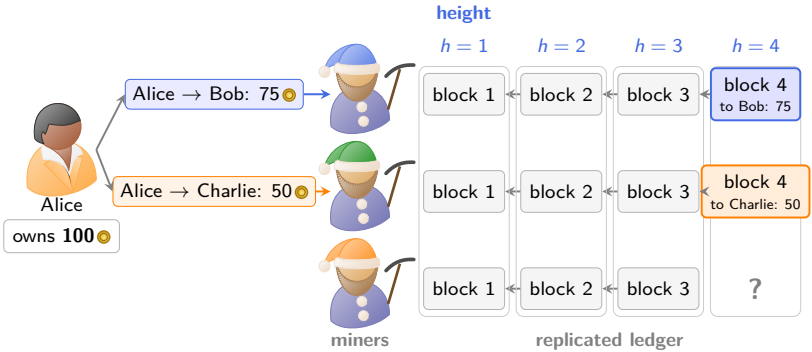
There is no single ledger: every node keeps its own copy, a **replica** of the ledger.



The **height** of a block is its position in the chain: the block at height h points to the block at height $h-1$.



Alice owns 100, yet she signs two transactions, 75 to Bob and 50 to Charlie, and sends them to two **different** miners. Together, they spend more than she owns: a **double spend**.



Miners **decide** which transaction comes first.

At each height h, the replicas agree on the block decided at h.

At each height h, the replicas **agree** on the block decided at h.

This is **consensus**.

	Nakamoto consensus (lecture 1)
system model	permissionless
assumption	majority of the computing power
agreement	probabilistic: a block is *very unlikely* to be reverted
cost	energy, latency of about an hour

	Nakamoto consensus (lecture 1)	Classical consensus (today)
system model	permissionless	permissioned: identified participants
assumption	majority of the computing power	fewer than half of the processes crash
agreement	probabilistic: a block is *very unlikely* to be reverted	deterministic: a decision is *final*
cost	energy, latency of about an hour	a few message delays

	Nakamoto consensus (lecture 1)	Classical consensus (today)
system model	permissionless	permissioned: identified participants
assumption	majority of the computing power	fewer than half of the processes crash
agreement	probabilistic: a block is *very unlikely* to be reverted	deterministic: a decision is *final*
cost	energy, latency of about an hour	a few message delays

- an example of classical consensus: Hyperledger Fabric (by Linux Foundation)



The General Assembly of the United Nations, in New York

How to *define* that we **agreed**?

Let $\mathscr{D}$ be some set of input values.

Let $\mathscr{D}$ be some set of input values.
A process *proposes* a value $u \in \mathscr{D}$ by calling `propose(u)`; it *decides* v when the call returns v.

(validity)	if a process decides u, then u was proposed
(agreement)	no two processes decide different values
(termination)	every (correct) process eventually decides

Let $\mathscr{D}$ be some set of input values.
A process *proposes* a value $u \in \mathscr{D}$ by calling `propose(u)`; it *decides* v when the call returns v.

(validity) ? if a process decides u, then u was proposed

(agreement) ? no two processes decide different values

(termination) ? every (correct) process eventually decides

For each property: **S** (safety) or **L** (liveness)?

S *safety*: nothing bad ever happens (a violation is visible in a finite prefix)
L *liveness*: something good eventually happens (lecture 2)

Let $\mathscr{D}$ be some set of input values.
A process *proposes* a value $u \in \mathscr{D}$ by calling `propose(u)`; it *decides* v when the call returns v.

(validity) **S** if a process decides u, then u was proposed

(agreement) **S** no two processes decide different values

(termination) **L** every (correct) process eventually decides

Validity and agreement are **safety** properties; termination is a **liveness** property.

Let $\mathscr{D}$ be some set of input values.
A process *proposes* a value $u \in \mathscr{D}$ by calling `propose(u)`; it *decides* v when the call returns v.

(validity) **S** if a process decides u, then u was proposed

(agreement) **S** no two processes decide different values

(termination) **L** every (correct) process eventually decides

Validity and agreement are **safety** properties; termination is a **liveness** property.

In a blockchain, $\mathscr{D}$ is the set of blocks (i.e., batches of transactions).

1 Context: consensus for blockchains

2 The FLP impossibility result

3 Paxos

4 Wrap-up

The model: two processes, read-write shared memory

- two processes, p and q, communicate through a **shared memory** of read-write registers

The model: two processes, read-write shared memory

- two processes, p and q, communicate through a **shared memory** of read-write registers
- as in lecture 2:
  - processes may *crash* (permanently stop computing)
  - they are *asynchronous*: arbitrarily slow
  - a process is *correct* when it never crashes

How to solve consensus when **no process fails**?

```
D ← null // shared register
```

```
D ← null // shared register
```

```
// at process p
function propose(u)
  D ← u
  return u
```

The easy case: no failures

```
D ← null // shared register
```

```
// at process p
function propose(u)
  D ← u
  return u
```

```
// at process q
function propose(u)
  wait for D ≠ null
  return D
```

The easy case: no failures

```
D ← null // shared register
```

```
// at process p
function propose(u)
  D ← u
  return u
```

```
// at process q
function propose(u)
  wait for D ≠ null
  return D
```

- p imposes its value; q waits for it

```
D ← null // shared register
```

```
// at process p
function propose(u)
  D ← u
  return u
```

```
// at process q
function propose(u)
  wait for D ≠ null
  return D
```

- p imposes its value; q waits for it
- if p is slow, or crashed, q **waits forever**

How to solve consensus when **one process can fail**?

Theorem (FLP, 1985)

Consensus is not solvable in an asynchronous distributed system when even a **single** process may fail.[†]

[†] M. J. Fischer et al. "Impossibility of Distributed Consensus with One Faulty Process". *Journal of the ACM* (1985).

Theorem (FLP, 1985)

Consensus is not solvable in an asynchronous distributed system when even a **single** process may fail.[†]

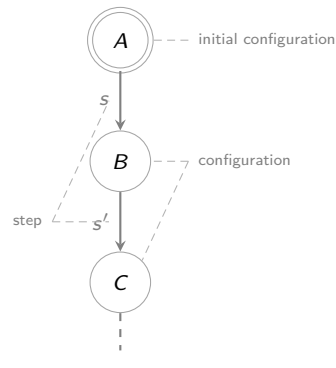
The proof that follows is for *binary* consensus, $\mathscr{D} = \{0, 1\}$, with our two processes p and q. It extends to any $\mathscr{D}$, to any number of processes, and to message passing.

[†] M. J. Fischer et al. "Impossibility of Distributed Consensus with One Faulty Process". *Journal of the ACM* (1985).

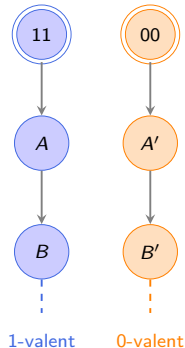
Proof setting: configurations

- a **configuration**: a global state of the system
- a **step**: an internal computation, then a read or write of the shared memory by some process
- a **run**: a sequence of steps

An algorithm defines a set of admissible runs.



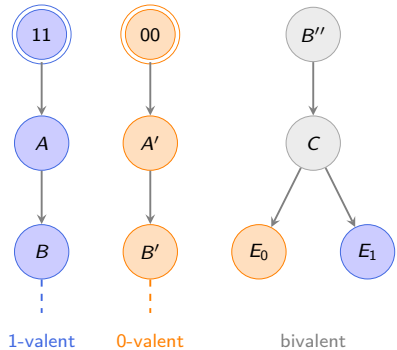
- a configuration is k-**valent** if from it only k can be decided; it is *monovalent*[†]
  - if all processes propose 1, the initial configuration is 1-valent (validity)



[†] H. Attiya et al. *Impossibility Results for Distributed Computing*. Morgan & Claypool, 2014.

Proof setting: valency

- a configuration is *k*-**valent** if from it only *k* can be decided; it is *monovalent*[†]
  - if all processes propose 1, the initial configuration is 1-valent (validity)
- a configuration is **bivalent** when both 0 and 1 can still be decided from it: it is not monovalent



[†] H. Attiya et al. *Impossibility Results for Distributed Computing*. Morgan & Claypool, 2014.

Claim 1 every run reaches a monovalent configuration

Claim 1 every run reaches a monovalent configuration

proof: eventually p and q decide, in some configuration C. By agreement they decide the same value, say 0, so all configurations after C are 0-valent.

Claim 1 **every run reaches a monovalent configuration**

Corollary **from a bivalent configuration, one must reach a *critical* configuration**: a bivalent configuration from which every step leads to a monovalent configuration

Claim 1 **every run reaches a monovalent configuration**

Corollary **from a bivalent configuration, one must reach a *critical* configuration**: a bivalent configuration from which every step leads to a monovalent configuration

proof: otherwise, from every bivalent configuration, some step leads to another bivalent configuration. Starting from C and repeating, we build a run $\langle C, D_0, D_1, \dots \rangle$ that contains only bivalent configurations: it never reaches a monovalent one, contradicting Claim 1.

Claim 1 **every run reaches a monovalent configuration**

Corollary **from a bivalent configuration, one must reach a *critical* configuration**: a bivalent configuration from which every step leads to a monovalent configuration

Claim 2 **there exists a bivalent initial configuration**

Claim 1 **every run reaches a monovalent configuration**

Corollary **from a bivalent configuration, one must reach a *critical* configuration**: a bivalent configuration from which every step leads to a monovalent configuration

Claim 2 **there exists a bivalent initial configuration**

proof: let p propose 0 and q propose 1. If p runs solo, it decides 0 (validity); if q runs solo, it decides 1. So both are reachable: the configuration is bivalent.

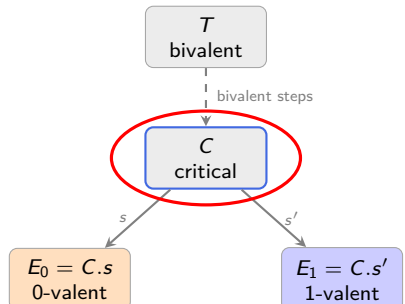
The critical configuration

Assume a working protocol. Let

- T be a bivalent initial configuration (Claim 2)
- C be a critical configuration reachable from T (Corollary)
- $E_0 = C.s$ be 0-valent, and $E_1 = C.s'$ be 1-valent

Processes are deterministic: s and s' are made by different processes, say p for s and q for s'.

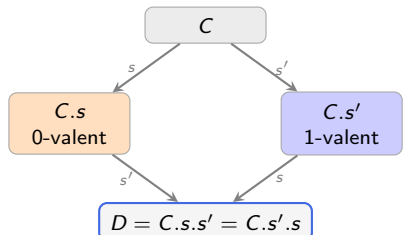
We derive a contradiction in each case.



Case 1: the steps access different registers

Let $reg(s)$ be the register accessed by s, and assume $reg(s) \neq reg(s')$.

- the two steps **commute**: $C.s.s' = C.s'.s$
- this configuration D is reachable from E_0, which is 0-valent, and from E_1, which is 1-valent
- contradiction: D is both 0-valent and 1-valent



Case 2: the steps access the same register

Let $x = reg(s) = reg(s')$.

Case 2: the steps access the same register

Let $x = reg(s) = reg(s')$.

- $s = r(x)$, $s' = w(x)$

Case 2: the steps access the same register

Let $x = reg(s) = reg(s')$.

- $s = r(x)$, $s' = w(x)$
  - s is a read by p: $C.s.s'$ and $C.s'$ are **identical for** q

Case 2: the steps access the same register

Let $x = reg(s) = reg(s')$.

- $s = r(x)$, $s' = w(x)$
  - s is a read by p: $C.s.s'$ and $C.s'$ are **identical for** q
  - q may run solo from $C.s.s'$ and decide 1: this contradicts that E_0 is 0-valent

Case 2: the steps access the same register

Let $x = reg(s) = reg(s')$.

- $s = r(x)$, $s' = w(x)$
  - s is a read by p: $C.s.s'$ and $C.s'$ are **identical for** q
  - q may run solo from $C.s.s'$ and decide 1: this contradicts that E_0 is 0-valent
- $s = w(x)$, $s' = r(x)$: symmetric

Case 2: the steps access the same register

Let $x = reg(s) = reg(s')$.

- $s = r(x)$, $s' = w(x)$
  - s is a read by p: $C.s.s'$ and $C.s'$ are **identical for** q
  - q may run solo from $C.s.s'$ and decide 1: this contradicts that E_0 is 0-valent
- $s = w(x)$, $s' = r(x)$: symmetric
- $s = w(x)$, $s' = w(x)$

Case 2: the steps access the same register

Let $x = reg(s) = reg(s')$.

- $s = r(x)$, $s' = w(x)$
  - s is a read by p: $C.s.s'$ and $C.s'$ are **identical for** q
  - q may run solo from $C.s.s'$ and decide 1: this contradicts that E_0 is 0-valent
- $s = w(x)$, $s' = r(x)$: symmetric
- $s = w(x)$, $s' = w(x)$
  - the write s' overwrites the write s: $C.s.s'$ and $C.s'$ are **identical for** q

Case 2: the steps access the same register

Let $x = reg(s) = reg(s')$.

- $s = r(x)$, $s' = w(x)$
  - s is a read by p: $C.s.s'$ and $C.s'$ are **identical for** q
  - q may run solo from $C.s.s'$ and decide 1: this contradicts that E_0 is 0-valent
- $s = w(x)$, $s' = r(x)$: symmetric
- $s = w(x)$, $s' = w(x)$
  - the write s' overwrites the write s: $C.s.s'$ and $C.s'$ are **identical for** q
  - q may run solo from $C.s.s'$ and decide 1: the same contradiction

Case 2: the steps access the same register

Let $x = reg(s) = reg(s')$.

- $s = r(x)$, $s' = w(x)$
  - s is a read by p: $C.s.s'$ and $C.s'$ are **identical for** q
  - q may run solo from $C.s.s'$ and decide 1: this contradicts that E_0 is 0-valent
- $s = w(x)$, $s' = r(x)$: symmetric
- $s = w(x)$, $s' = w(x)$
  - the write s' overwrites the write s: $C.s.s'$ and $C.s'$ are **identical for** q
  - q may run solo from $C.s.s'$ and decide 1: the same contradiction

There is no critical configuration, hence no correct protocol. □

It says

- no deterministic algorithm guarantees *both* safety and termination in an asynchronous system with one crash

It says

- no deterministic algorithm guarantees *both* safety and termination in an asynchronous system with one crash

- validity
- agreement
- termination

Which property would you **give up**?

It says

- no deterministic algorithm guarantees *both* safety and termination in an asynchronous system with one crash

- validity
- agreement
- ~~termination~~

Which property would you **give up**?

What FLP says, and what it does not

It says

- no deterministic algorithm guarantees *both* safety and termination in an asynchronous system with one crash

It does not say

- that consensus is impossible to reach in practice

What FLP says, and what it does not

It says

- no deterministic algorithm guarantees *both* safety and termination in an asynchronous system with one crash

It does not say

- that consensus is impossible to reach in practice

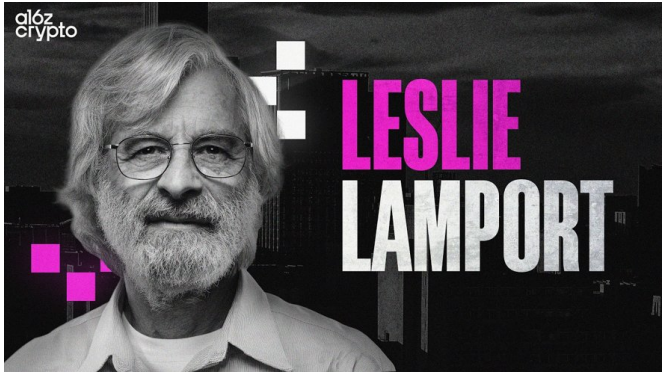
$\rightarrow$ next section: **Paxos**, always safe, and terminates when the system is not asynchronous

1 Context: consensus for blockchains

2 The FLP impossibility result

3 Paxos

4 Wrap-up



"Leslie Lamport on the Science of Distributed Systems", a16z Frontier

watch the video, (17'49 → 19'47)

- n processes, linked by **reliable channels**, as in lecture 2
  - processes may *crash*, and they are *asynchronous*

- n processes, linked by **reliable channels**, as in lecture 2
  - processes may *crash*, and they are *asynchronous*
- at most $f < n/2$ processes crash: a **majority** is correct (why this is needed: exercises)

- n processes, linked by **reliable channels**, as in lecture 2
  - processes may *crash*, and they are *asynchronous*
- at most $f < n/2$ processes crash: a **majority** is correct (why this is needed: exercises)
- running example: $n = 5$ processes, $p_1, \ldots, p_5$, and $f = 2$

How to solve consensus with **messages**, when a minority of the processes may crash?

First idea: a fixed leader

- a **leader** sends its proposal to all; every process decides the value it receives

First idea: a fixed leader

- a **leader** sends its proposal to all; every process decides the value it receives
- if the leader crashes, the others **wait forever**: the easy case again

First idea: a fixed leader

- a **leader** sends its proposal to all; every process decides the value it receives
- if the leader crashes, the others **wait forever**: the easy case again
- so the leader must **change**, with a leader election Ω
  - at each process, Ω returns a process, the current leader
  - eventually, Ω returns the same correct process everywhere
  - before that, several processes may believe that they lead

Second idea: changing leader

- each attempt of a leader has a **ballot** b: p_i uses the ballots $b \equiv i \pmod{n}$



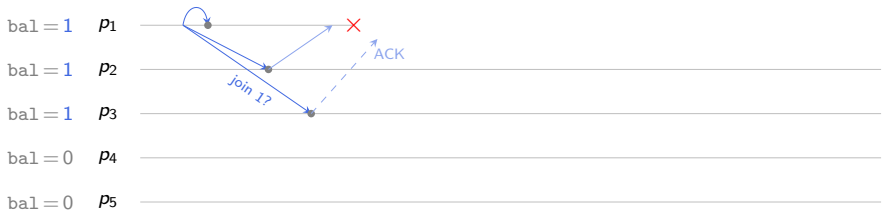
Second idea: changing leader

- each attempt of a leader has a **ballot** b: p_i uses the ballots $b \equiv i \pmod{n}$
- the leader asks to **join** b; a process that joins answers **ACK**



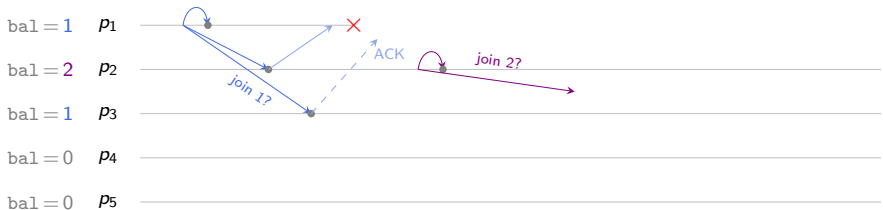
Second idea: changing leader

- each attempt of a leader has a **ballot** b: p_i uses the ballots $b \equiv i \pmod{n}$
- the leader asks to **join** b; a process that joins answers **ACK**



Second idea: changing leader

- each attempt of a leader has a **ballot** b: p_i uses the ballots $b \equiv i \pmod{n}$
- the leader asks to **join** b; a process that joins answers **ACK**



Second idea: changing leader

- each attempt of a leader has a **ballot** b: p_i uses the ballots $b \equiv i \pmod{n}$
- the leader asks to **join** b; a process that joins answers **ACK**



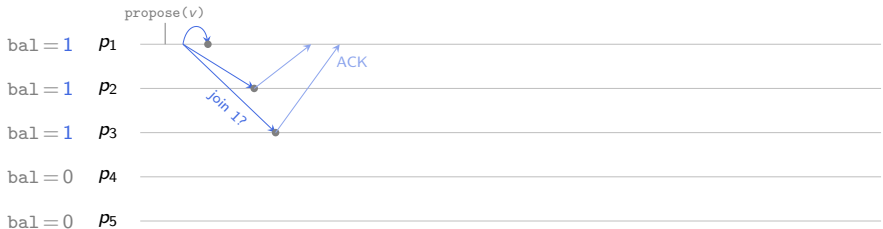
Second idea: changing leader

- each attempt of a leader has a **ballot** b: p_i uses the ballots $b \equiv i \pmod{n}$
- the leader asks to **join** b; a process that joins answers **ACK**
- a process that joined a higher ballot does not join, and answers **NACK**



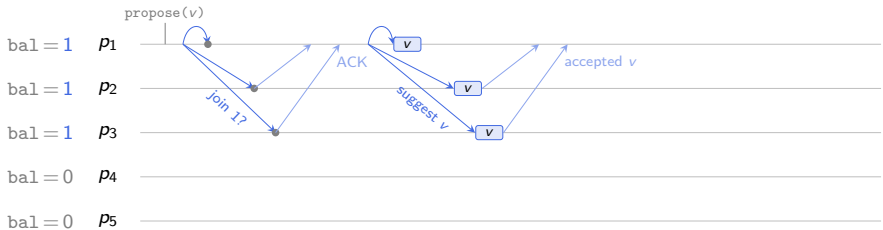
Third idea: decide on a quorum

- once the leader of ballot b knows that a **quorum** (a majority) has joined b, it **suggests** a proposal



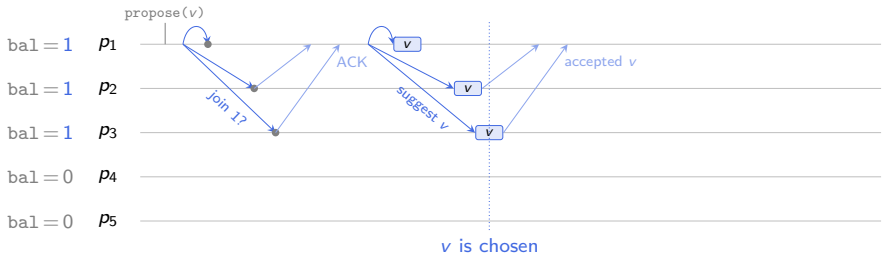
Third idea: decide on a quorum

- once the leader of ballot b knows that a **quorum** (a majority) has joined b, it **suggests** a proposal
- a process **accepts** it, unless it joined a higher ballot, and tells the ballot leader



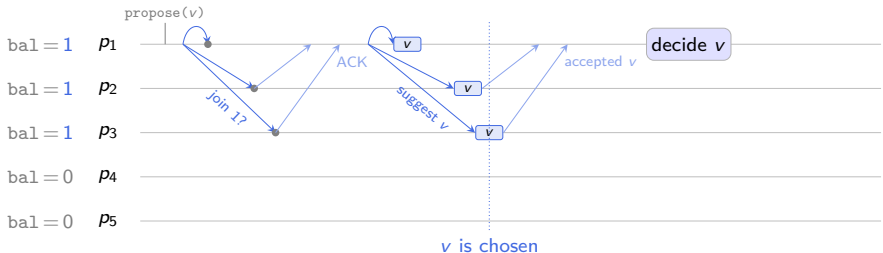
Third idea: decide on a quorum

- once the leader of ballot b knows that a **quorum** (a majority) has joined b, it **suggests** a proposal
- a process **accepts** it, unless it joined a higher ballot, and tells the ballot leader
- a value is **chosen** once a quorum accepts it; a process decides value v when it knows that v is chosen



Third idea: decide on a quorum

- once the leader of ballot b knows that a **quorum** (a majority) has joined b, it **suggests** a proposal
- a process **accepts** it, unless it joined a higher ballot, and tells the ballot leader
- a value is **chosen** once a quorum accepts it; a process decides value v when it knows that v is chosen



Paxos main invariant

If u is **chosen** at ballot b, and v is suggested at a ballot $b' \geq b$, then $u = v$.

If u is **chosen** at ballot b, and v is suggested at a ballot $b' \geq b$, then $u = v$.

To keep it, the leader of b' must know which value *may have been chosen at a lower ballot*. For this, each process keeps two more variables:

If u is **chosen** at ballot b, and v is suggested at a ballot $b' \geq b$, then $u = v$.

To keep it, the leader of b' must know which value *may have been chosen at a lower ballot*. For this, each process keeps two more variables:

- `vbal`: the ballot of the last value it **accepted**

If u is **chosen** at ballot b, and v is suggested at a ballot $b' \geq b$, then $u = v$.

To keep it, the leader of b' must know which value *may have been chosen at a lower ballot*. For this, each process keeps two more variables:

- `vbal`: the ballot of the last value it **accepted**
- `val`: that value

If u is **chosen** at ballot b, and v is suggested at a ballot $b' \geq b$, then $u = v$.

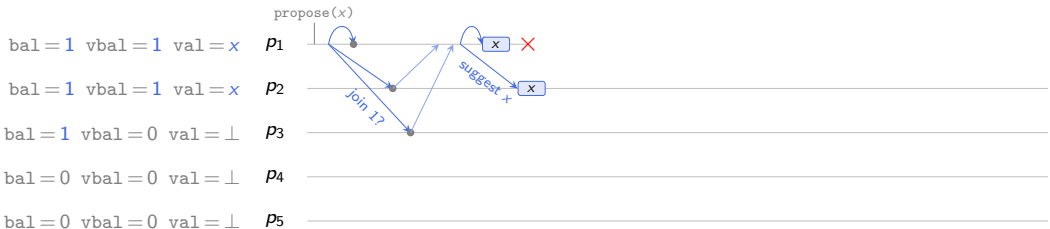
To keep it, the leader of b' must know which value *may have been chosen at a lower ballot*. For this, each process keeps two more variables:

- `vbal`: the ballot of the last value it **accepted**
- `val`: that value

The leader collects them from a quorum, which intersects every quorum that chose a value, and suggests the value with the **highest** `vbal`.

Paxos main invariant

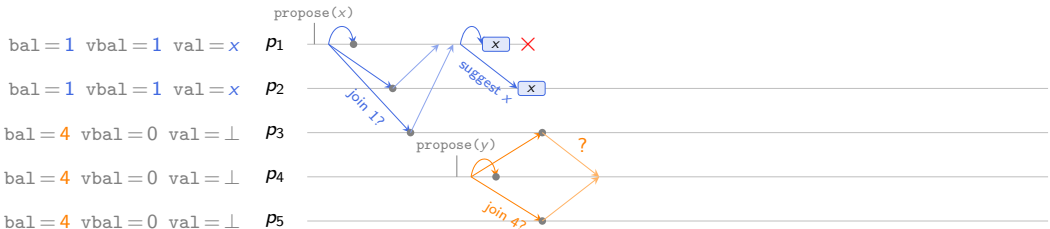
If u is **chosen** at ballot b, and v is suggested at a ballot $b' \geq b$, then $u = v$.



Paxos main invariant

If u is **chosen** at ballot b, and v is suggested at a ballot $b' \geq b$, then $u = v$.

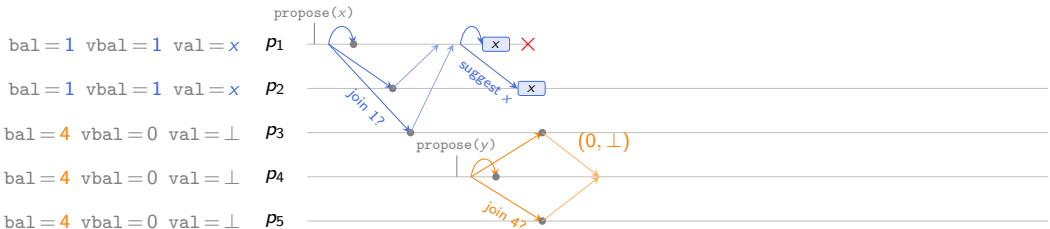
- **phase 1** (*prepare*): each ACK **piggybacks** the last accepted value, with its ballot



Paxos main invariant

If u is **chosen** at ballot b, and v is suggested at a ballot $b' \geq b$, then $u = v$.

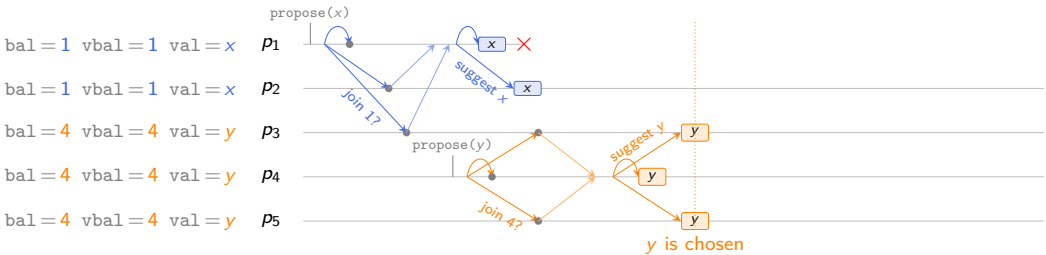
- **phase 1** (*prepare*): each ACK **piggybacks** the last accepted value, with its ballot



Paxos main invariant

If u is **chosen** at ballot b, and v is suggested at a ballot $b' \geq b$, then $u = v$.

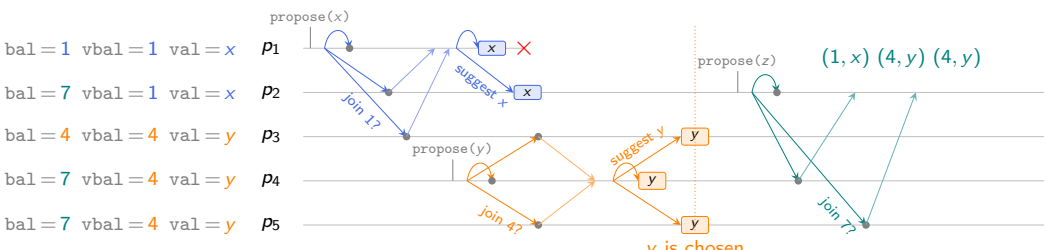
- **phase 1** (*prepare*): each ACK **piggybacks** the last accepted value, with its ballot
- **phase 2** (*accept*): the leader suggests the value with the **highest** ballot in the ACKs (if none, its proposal)



Paxos main invariant

If u is **chosen** at ballot b, and v is suggested at a ballot $b' \geq b$, then $u = v$.

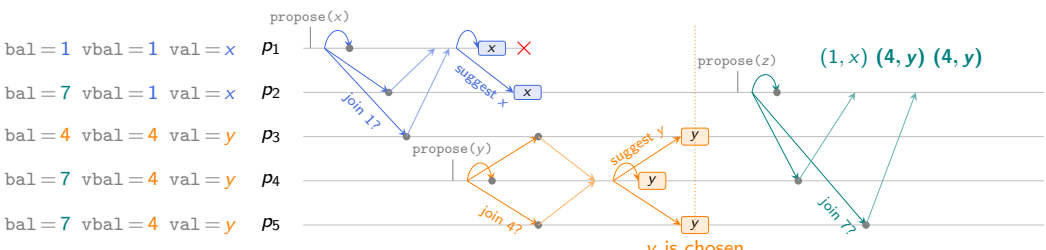
- **phase 1** (*prepare*): each ACK **piggybacks** the last accepted value, with its ballot
- **phase 2** (*accept*): the leader suggests the value with the **highest** ballot in the ACKs (if none, its proposal)



Paxos main invariant

If u is **chosen** at ballot b, and v is suggested at a ballot $b' \geq b$, then $u = v$.

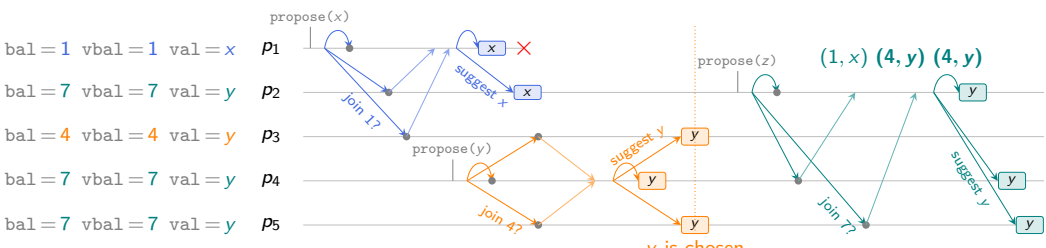
- **phase 1** (*prepare*): each ACK **piggybacks** the last accepted value, with its ballot
- **phase 2** (*accept*): the leader suggests the value with the **highest** ballot in the ACKs (if none, its proposal)



Paxos main invariant

If u is **chosen** at ballot b, and v is suggested at a ballot $b' \geq b$, then $u = v$.

- **phase 1** (*prepare*): each ACK **piggybacks** the last accepted value, with its ballot
- **phase 2** (*accept*): the leader suggests the value with the **highest** ballot in the ACKs (if none, its proposal)



Paxos: the code at process p

Variables, and Ω

`bal` $\leftarrow$ 0: the highest ballot joined; `vbal` $\leftarrow$ 0: the ballot of the last accepted value; `val` $\leftarrow$ null: that value; `my_suggestion` $\leftarrow$ null (local): the proposal of p. Ω returns the current leader: eventually, the same correct process everywhere.

prepare (phase 1) 1A: join b?, 1B: ACK

```
propose(u)
  my_suggestion ← u
```

```
phase1A(b)
  pre: p = Ω, b ≥ 1,
       my_suggestion ≠ null
  broadcast ⟨1A, b⟩
```

```
phase1B
  pre: recv ⟨1A, b⟩ from q, bal < b
  bal ← b
  send ⟨1B, b, vbal, val⟩ to q
```

```
/* each box is an action;
 * an action is enabled when
 *   its precondition (pre:) holds;
 * the process repeatedly runs an
 *   enabled action, atomically;
 * recv m: m was received
 */
```

Paxos: the code at process p

Variables, and Ω

`bal` $\leftarrow$ 0: the highest ballot joined; `vbal` $\leftarrow$ 0: the ballot of the last accepted value; `val` $\leftarrow$ null: that value; `my_suggestion` $\leftarrow$ null (local): the proposal of p. Ω returns the current leader: eventually, the same correct process everywhere.

prepare (phase 1) 1A: join b?, 1B: ACK

```
propose(u)
  my_suggestion ← u
```

```
phase1A(b)
  pre: p = Ω, b ≥ 1,
       my_suggestion ≠ null
  broadcast ⟨1A, b⟩
```

```
phase1B
  pre: recv ⟨1A, b⟩ from q, bal < b
  bal ← b
  send ⟨1B, b, vbal, val⟩ to q
```

accept (phase 2) 2A: suggest, 2B: accepted

```
phase2A(b)
  pre: recv ⟨1B, b, b', u⟩ from quorum Q
  Γ ← max {b' : recv ⟨1B, b, b', _⟩} ∪ {0}
  if Γ = 0 then v ← my_suggestion
  else v: recv ⟨1B, b, Γ, v⟩
  broadcast ⟨2A, b, v⟩
```

```
phase2B(b, v)
  pre: recv ⟨2A, b, v⟩, bal ≤ b
  bal, vbal ← b; val ← v
  broadcast ⟨2B, b, v⟩
```

```
decide(v)  pre: recv ⟨2B, b, v⟩ from quorum Q
```

- **validity**: a leader suggests either its proposal (`my_suggestion`), or a value reported in an ACK, which was accepted, hence suggested before
  - by induction, every suggested value is a proposal; a decided value was chosen, hence suggested

- **validity**: a leader suggests either its proposal (`my_suggestion`), or a value reported in an ACK, which was accepted, hence suggested before
  - by induction, every suggested value is a proposal; a decided value was chosen, hence suggested
- **agreement**: if u and v are decided, they are chosen at some ballots $b \leq b'$
  - v was suggested at $b' \geq b$: by the Paxos main invariant, $u = v$

Why is Paxos safe?

- **validity**: a leader suggests either its proposal (`my_suggestion`), or a value reported in an ACK, which was accepted, hence suggested before
  - by induction, every suggested value is a proposal; a decided value was chosen, hence suggested
- **agreement**: if u and v are decided, they are chosen at some ballots $b \leq b'$
  - v was suggested at $b' \geq b$: by the Paxos main invariant, $u = v$

Paxos main invariant

If u is **chosen** at ballot b, and v is suggested at a ballot $b' \geq b$, then $u = v$. (Its proof: exercises.)

Paxos: liveness

- safety holds in **every** execution: asynchrony, crashes, several leaders at once

Paxos: liveness

- safety holds in **every** execution: asynchrony, crashes, several leaders at once
- two leaders may keep preempting each other with higher ballots, and no value is ever decided (consistent with FLP)

Paxos: liveness

- safety holds in **every** execution: asynchrony, crashes, several leaders at once
- two leaders may keep preempting each other with higher ballots, and no value is ever decided (consistent with FLP)
- with the leader election Ω, one leader eventually runs alone, and every correct process decides

The mapping

- value = a block (or a batch of transactions)
- leader = block proposer
- quorum = a majority of miners

The mapping

- value = a block (or a batch of transactions)
- leader = block proposer
- quorum = a majority of miners

The limits

- crash failures only: miners may also be **Byzantine** (lecture 4)
- one instance decides *one* value; a chain needs a **sequence** of instances (Multi-Paxos, lecture 5)
- permissioned: the set of miners is known

1. Context: consensus for blockchains
2. The FLP impossibility result
3. Paxos
4. Wrap-up

- **Nakamoto consensus** sidesteps FLP
  - termination is only probabilistic: no deterministic finality
  - it assumes that blocks propagate fast relative to the block interval (10 minutes): a timing assumption

- **Nakamoto consensus** sidesteps FLP
  - termination is only probabilistic: no deterministic finality
  - it assumes that blocks propagate fast relative to the block interval (10 minutes): a timing assumption
- **classical protocols**, such as Paxos, assume **partial synchrony**: the system eventually behaves synchronously, at an unknown time[†]
  - safety holds in *every* execution
  - liveness holds once the network behaves, and a correct leader emerges

[†] C. Dwork et al. "Consensus in the Presence of Partial Synchrony". *Journal of the ACM* (1988).

- **Nakamoto consensus** sidesteps FLP
  - termination is only probabilistic: no deterministic finality
  - it assumes that blocks propagate fast relative to the block interval (10 minutes): a timing assumption
- **classical protocols**, such as Paxos, assume **partial synchrony**: the system eventually behaves synchronously, at an unknown time[†]
  - safety holds in *every* execution
  - liveness holds once the network behaves, and a correct leader emerges
- a trade-off: permissionless and probabilistic, or permissioned and deterministic

[†] C. Dwork et al. "Consensus in the Presence of Partial Synchrony". *Journal of the ACM* (1988).

- **Nakamoto consensus** sidesteps FLP
  - termination is only probabilistic: no deterministic finality
  - it assumes that blocks propagate fast relative to the block interval (10 minutes): a timing assumption
- **classical protocols**, such as Paxos, assume **partial synchrony**: the system eventually behaves synchronously, at an unknown time[†]
  - safety holds in *every* execution
  - liveness holds once the network behaves, and a correct leader emerges
- a trade-off: permissionless and probabilistic, or permissioned and deterministic
- in practice: Bitcoin (probabilistic); Cosmos and Aptos (partial synchrony); Ethereum (both: partial synchrony for finality, a probabilistic fork choice)

[†] C. Dwork et al. "Consensus in the Presence of Partial Synchrony". *Journal of the ACM* (1988).

Takeaways

- a blockchain must **agree** on the next block and on the order of transactions: **consensus**

Takeaways

- a blockchain must **agree** on the next block and on the order of transactions: **consensus**
- **FLP**: no deterministic algorithm solves consensus in an asynchronous system with one crash; the proof is a *valency* argument (bivalent, critical configuration)

Takeaways

- a blockchain must **agree** on the next block and on the order of transactions: **consensus**
- **FLP**: no deterministic algorithm solves consensus in an asynchronous system with one crash; the proof is a *valency* argument (bivalent, critical configuration)
- in practice: **partial synchrony**, $f < n/2$, a **leader**; Nakamoto consensus trades determinism for probabilistic termination

Takeaways

- a blockchain must **agree** on the next block and on the order of transactions: **consensus**
- **FLP**: no deterministic algorithm solves consensus in an asynchronous system with one crash; the proof is a *valency* argument (bivalent, critical configuration)
- in practice: **partial synchrony**, $f < n/2$, a **leader**; Nakamoto consensus trades determinism for probabilistic termination
- **Paxos**: a value is chosen once a **quorum** accepts it, and two quorums intersect; **ballots** order the leaders

Takeaways

- a blockchain must **agree** on the next block and on the order of transactions: **consensus**
- **FLP**: no deterministic algorithm solves consensus in an asynchronous system with one crash; the proof is a *valency* argument (bivalent, critical configuration)
- in practice: **partial synchrony**, $f < n/2$, a **leader**; Nakamoto consensus trades determinism for probabilistic termination
- **Paxos**: a value is chosen once a **quorum** accepts it, and two quorums intersect; **ballots** order the leaders
- phase 1 (prepare): a quorum **joins** the ballot and reports its accepted values; phase 2 (accept): the leader **suggests** the value of the highest ballot; safe always, live with a leader

- **lecture 4**: Byzantine failures
- **lecture 5**: replicated state machines — a *sequence* of consensus instances

Readings

- Fischer, Lynch, and Paterson, *Impossibility of distributed consensus with one faulty process* [†]
- Dwork, Lynch, and Stockmeyer, *Consensus in the presence of partial synchrony* [‡]
- Lamport, *The part-time parliament* [§]
- Ongaro and Ousterhout, *In search of an understandable consensus algorithm* (Raft) [¶]

[†] M. J. Fischer et al. "Impossibility of Distributed Consensus with One Faulty Process". *Journal of the ACM* (1985).

[‡] C. Dwork et al. "Consensus in the Presence of Partial Synchrony". *Journal of the ACM* (1988).

[§] L. Lamport. "The Part-Time Parliament". *ACM Transactions on Computer Systems* (1998).

[¶] D. Ongaro et al. "In Search of an Understandable Consensus Algorithm". *USENIX ATC*. 2014.